# Chapter 2. Binary images

**Contents**

# Chapter 2. Binary images

## 2.1 Introduction

Binary images are the simplest type of images used in computer vision because pixels have one of two values: 0 (background) or 255 (object or foreground).

They are very easy to process, store, and analyze.
They are very useful — there are many vision tasks, especially in structured environments such as assembly lines in factories, that can be performed efficiently and robustly using binary images.

# Chapter 2. Binary images

## 2.1 Introduction

**How to obtain a binary image?**

We threshold a gray level image (0-255) with a value T.
The result is a function, b(x,y), for which the output value is 0 if the corresponding value in the gray level image is less than the threshold and 1 otherwise (it can also be the other way around).

$$b(x,y) = \begin{cases} 0, & g(x,y) < T \\ 1, & g(x,y) \geq T \end{cases}$$

# Chapter 2. Binary images

## 2.1 Introduction

How to obtain the threshold?

In order to select the threshold T, we can compute the histogram of the original gray-level image.

The histogram seen here has two peaks, one corresponding to the background and the other to the objects in the foreground.

The best choice for the threshold T would be a value that lies in the valley between the
two picks.

# Chapter 2. Binary images

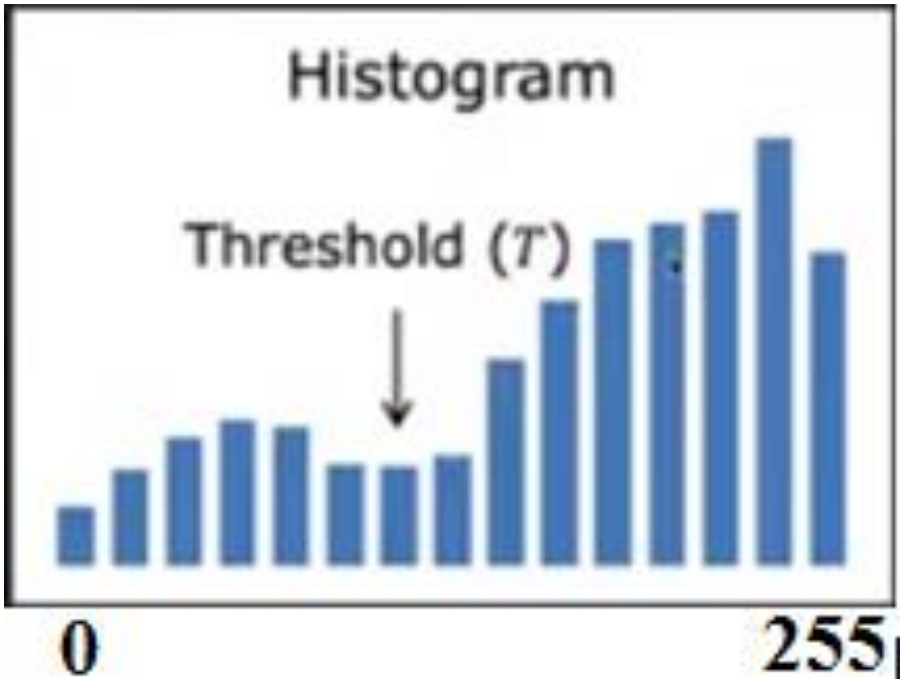
## 2.1 Introduction

How to obtain the threshold?

In order to select the threshold T, we can compute the histogram of the original gray-level image.

The histogram seen here has two peaks, one corresponding to the background and the other to the objects in the foreground.

The best choice for the threshold T would be a value that lies in the valley between the two picks.



# Chapter 2. Binary images

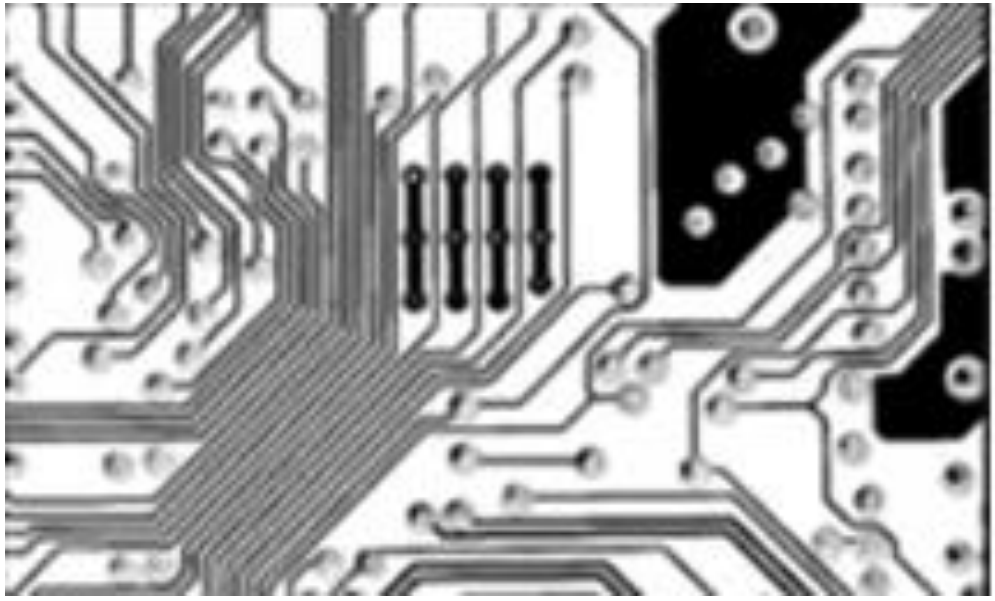
## 2.1 Introduction

How to obtain the threshold?

In order to select the threshold T, we can compute the histogram of the original gray-level image.

The histogram seen here has two peaks, one corresponding to the background and the other to the objects in the foreground.

The best choice for the threshold T would be a value that lies in the valley between the two picks.

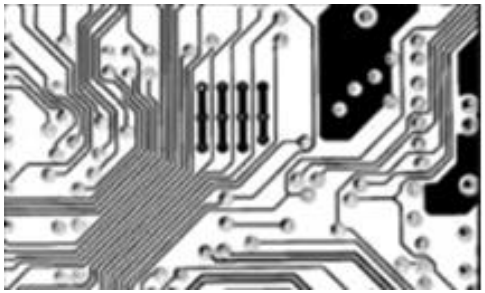


# Chapter 2. Binary images

## 2.1 Introduction

**Applications of binary images**

Detecting defects in printed circuit boards, fingerprint analysis, detection and decoding of visual codes like QR codes and medical image analysis.



# Chapter 2. Binary images

## 2.1 Introduction

**Applications of binary images**

In any given stable configuration, the 3D object produces a 2D shape that may be translated or rotated in the image.

We can apply an algorithm to recognize a 2D shape in order to recognize the corresponding 3D object.

# Chapter 2. Binary images

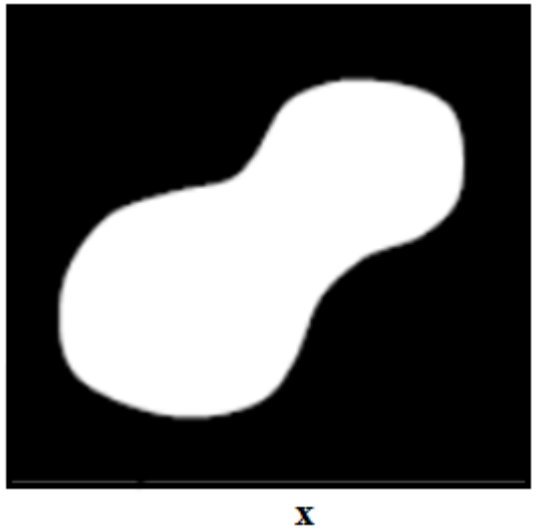
## 2.2 Geometric Properties

# Chapter 2. Binary images

## 2.2 Geometric Properties

The binary image is continuous with spatial coordinates *x* and *y*.

We define the characteristic function *b(x,y)* is 1 for points on the object and 0 for points in the background.

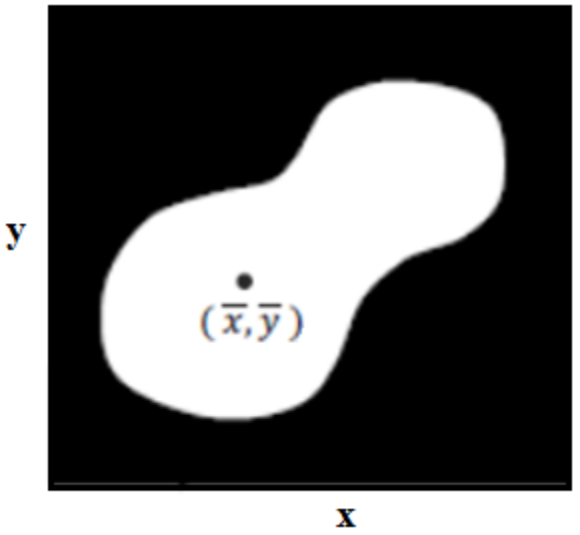


# Chapter 2. Binary images

## 2.2 Geometric Properties

**Geometric property**: Area of the object, which is the zeroth moment.
The area is computed by integrating the characteristic function b(x,y) over the entire image.

The area is useful property because it allows distinguishing between objects.



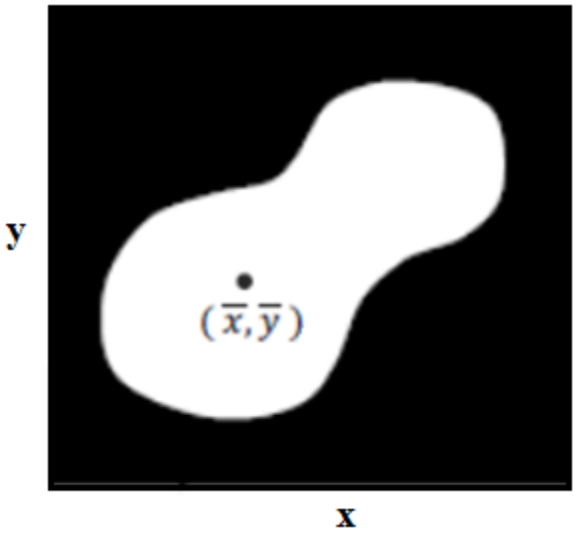

Area: (Zeroth Moment)

$$A = \iint_I b(x,y)\,dx\,dy$$

# Chapter 2. Binary images

## 2.2 Geometric Properties

**Geometric property**: Location of the object, center of the area computed as the first moment.



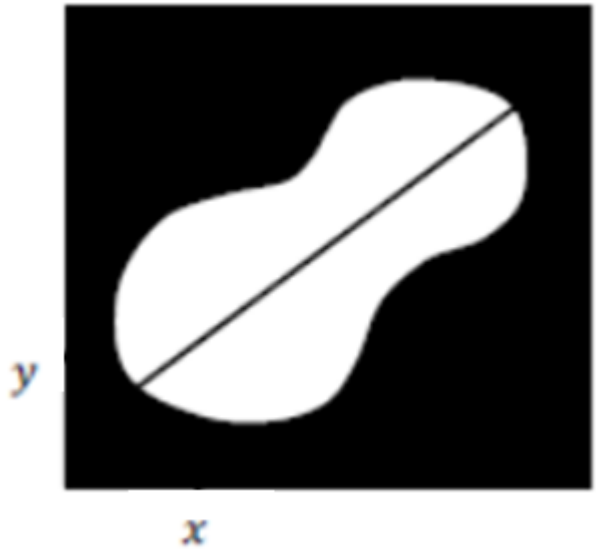

Position: Center of Area (First Moment)

$$\bar{x} = \frac{1}{A}\iint_I x\, b(x,y)\, dx\, dy \quad , \quad \bar{y} = \frac{1}{A}\iint_I y\, b(x,y)\, dx\, dy$$

# Chapter 2. Binary images

## 2.2 Geometric Properties

**Geometric property**: Orientation of the object.

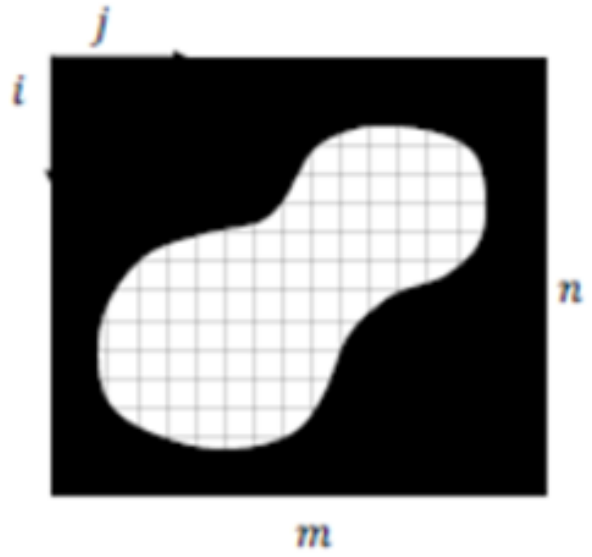


# Chapter 2. Binary images

## 2.2 Geometric Properties

**Geometric property**:

In the case of **discrete binary images**, the area of the object A is just the sum of all pixel values in the image.




The expressions for computing the area (the zeroth moment) and the coordinates of the center (the first moments) in the case of a discrete image are given by the following expressions, where $b_{ij}$ is the value of the pixel (i,j):

Area: $$A = \sum_{i=1}^{n}\sum_{j=1}^{m} b_{ij}$$

Position: Center of Area (First Moment)

$$\overline{x} = \frac{1}{A}\sum_{i=1}^{n}\sum_{j=1}^{m} i b_{ij} \qquad \overline{y} = \frac{1}{A}\sum_{i=1}^{n}\sum_{j=1}^{m} j b_{ij}$$

# Chapter 2. Binary images

## 2.3 Segmenting Binary Images

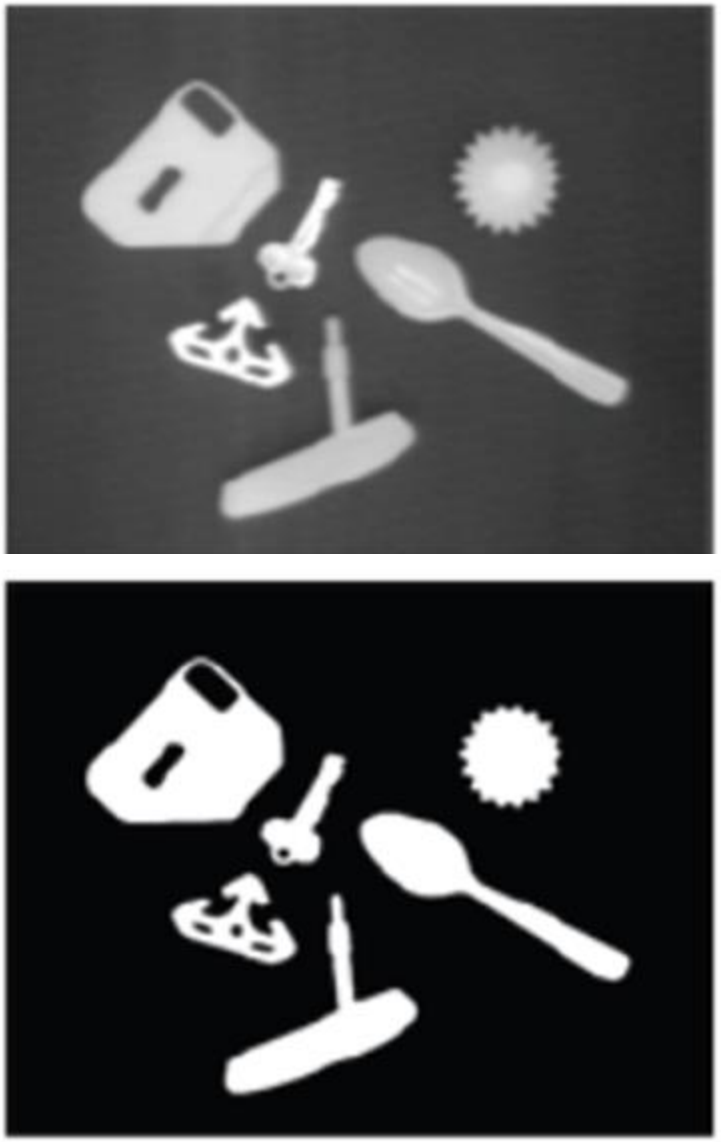
# Chapter 2. Binary images

## 2.3 Segmenting Binary Images

Segmenting binary images into regions that correspond to different objects.

An image can have multiple objects, we need then to label each object with a unique label before we compute its geometric properties.

We assume in the first that objects do not overlap or even touch each other — that is, each object is surrounded by the background.



# Chapter 2. Binary images

## 2.3 Segmenting Binary Images

Two points, A and B, are considered to be "connected" if there exists a path from A to B along which the characteristic function of the binary image is constant.
A connected component in a binary image is a maximal set of such connected points.

A and B are connected if path exists between A and B along which b(x,y) is constant.
.

# Chapter 2. Binary images

## 2.3 Segmenting Binary Images

The algorithm starts by scanning the image, each row is scanned from left to right and the rows are scanned from top to bottom until it locates an unlabeled point with *b*=1. If such a point is not found, the algorithm terminates.


**Algorithm**

Region Growing Algorithm

(a) Find Unlabeled "Seed" point with $b = 1$.
If not found, Terminate.

(b) Assign New Label to seed point

(c) Assign Same Label to its Neighbors with $b = 1$

(d) Assign Same Label to Neighbors of Neighbors with $b = 1$. Repeat until no more Unlabeled Neighbors with b=1.

(e) Go to (a)

# Chapter 2. Binary images

## 2.3 Segmenting Binary Images

The point is assigned a new label and becomes the seed point for a new region.
This label is then assigned to all the neighbors of the seed point that have b=1, to the neighbors of those neighbors, and so on.

This iterative process grows a region with the same label until there are no more unlabeled neighbors with b=1.
When it reaches that point, the algorithm returns to the first step, which is to continue scanning for the next unlabeled seed point with b=1.
.

**Algorithm**

Region Growing Algorithm

(a) Find Unlabeled "Seed" point with $b = 1$.
If not found, Terminate.

(b) Assign New Label to seed point

(c) Assign Same Label to its Neighbors with $b = 1$

(d) Assign Same Label to Neighbors of Neighbors with $b = 1$. Repeat until no more Unlabeled Neighbors with b=1.

(e) Go to (a)

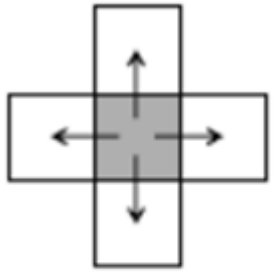
# Chapter 2. Binary images

## 2.3 Segmenting Binary Images

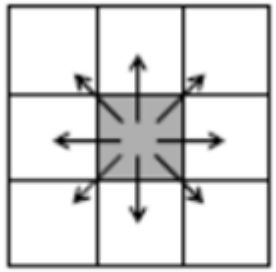
**Two kind of neighbors:**

The first is based on 4-Connectedness, in which a pixel's neighbors are the four pixels to the left, right, top, and bottom of it.

The other is 8-Connectedness (right), which includes the diagonal pixels.



4-Connectedness
4-C

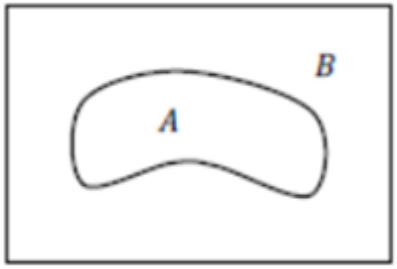


8-Connectedness
8-C

# Chapter 2. Binary images

## 2.3 Segmenting Binary Images

**Problems with the two neighbourhood:**




Jordan's Curve Theorem.
It states that a closed curve must divide a region up into two connected regions. In the figure, A and B are separate, fully connected regions that are not connected to each other.

Both 4-Connectedness and 8-Connectedness violate Jordan's Curve Theorem.
Consider the binary image on the right. We assume that the **0** in the outer ring of the image are connected to each other as the image shown here sits within a larger image with only 0s.

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 |

Binary image

# Chapter 2. Binary images

## 2.3 Segmenting Binary Images

In the case of 4-Connectedness, none of the 1s are connected to each other because this definition neglects the diagonals.

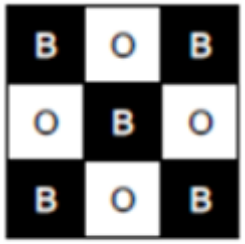
As a result, we obtain four separate objects (O1-O4). However, we do end up with two backgrounds, B1 and B2. So, we end up with four disconnected objects and two disconnected backgrounds, which violates Jordan's theorem.


Hull with closed loop

# Chapter 2. Binary images

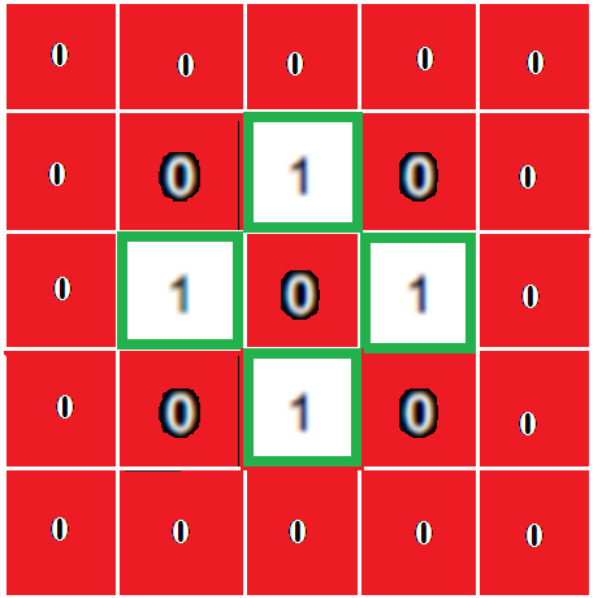
## 2.3 Segmenting Binary Images

With 8-Connectedness (right), the four 1s are grouped into one object with the shape of a ring. However, in this case, the background inside the ring is connected to the background outside it, which again violates Jordan's theorem.




8-C

Connected background with closed loop



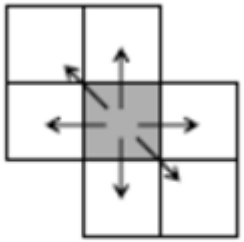
# Chapter 2. Binary images

## 2.3 Segmenting Binary Images

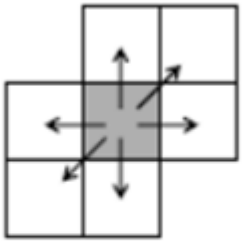
**Solution to this problem**:

**Asymmetry** into the definition of a neighbour: **The 6-Connectedness**.

Note that in each of these definitions, two diagonal pixels are dropped from the neighborhood.



6-Connectedness
6-C



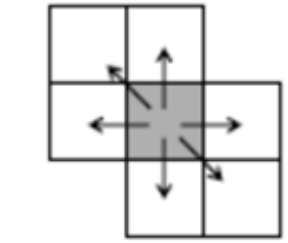
6-Connectedness
6-C

# Chapter 2. Binary images

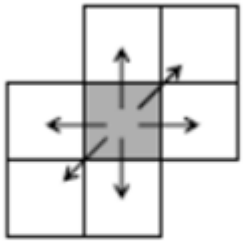
## 2.3 Segmenting Binary Images

**Solution to this problem:**

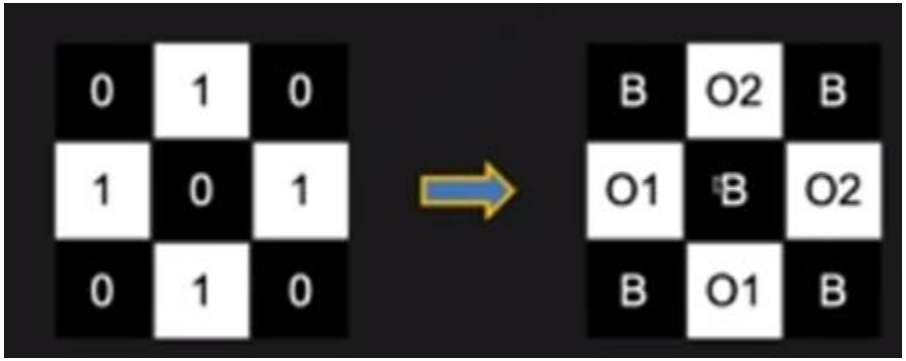
Returning to our example image patch, we now obtain two line segments and a single connected background. This outcome is consistent with Jordan's Curve Theorem.



6-Connectedness
6-C



6-Connectedness
6-C

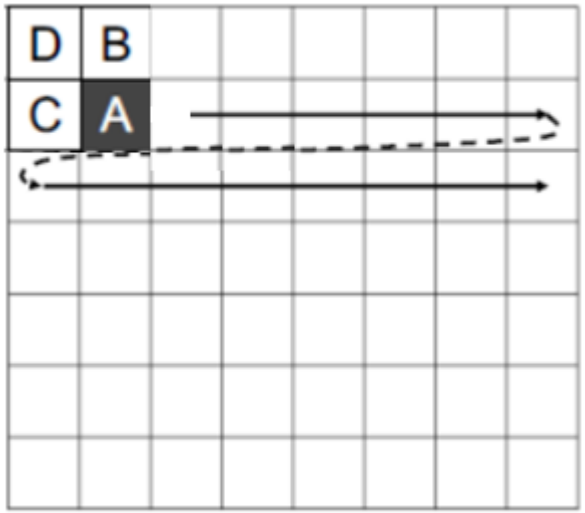


# Chapter 2. Binary images

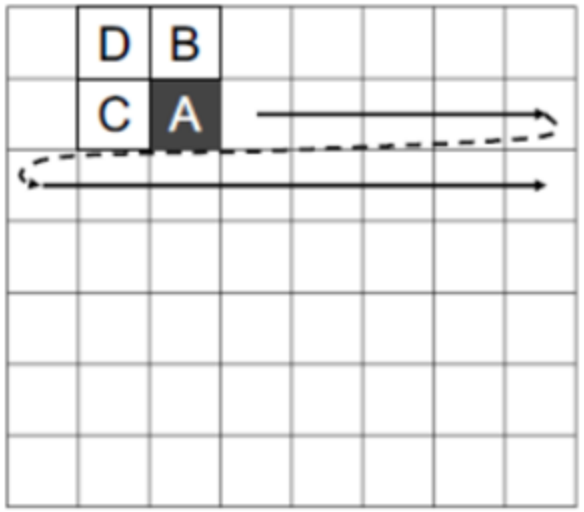
## 2.3 Segmenting Binary Images

**Second algorithm: Sequential labeling**

We raster scan the image. We wish to label the pixel A. Since we are raster scanning the image, when we arrive at pixel A, we already have labelled pixels B, C, and D.




It is worth noting that, by using only the top, diagonal, and left neighbours to label a pixel, we are implicitly introducing the asymmetry needed to satisfy Jordan's theorem.

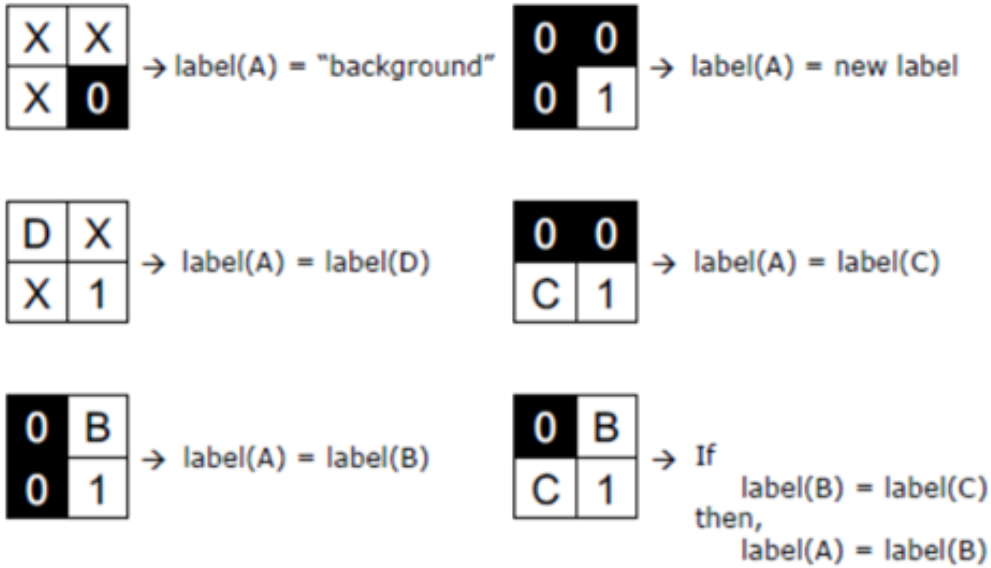


# Chapter 2. Binary images

## 2.3 Segmenting Binary Images

### Second algorithm: Sequential labeling

Here is how sequential labeling works.
If A is 0, we will call it part of the background, irrespective of what the labels of pixels B, C, and D are. If A is 1 and the three neighbors are 0, we give it a new label.
If A is a 1 and D is labeled, we will give A the label of D, irrespective of the values of B and C.

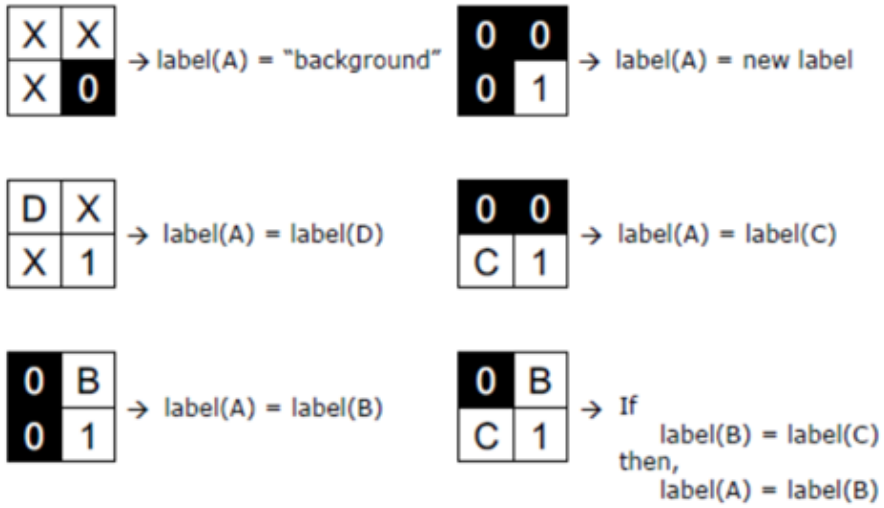


# Chapter 2. Binary images

## 2.3 Segmenting Binary Images

### Second algorithm: Sequential labeling

If A is 1, B and D are 0, and C is labeled, then A is given the label of C. Similarly, if C and D are 0, but B is labeled, then A is given the label of B. Now consider the case where A is 1, B and C have been labeled, and D is a background. If B and C have the same label, then A is given that label.

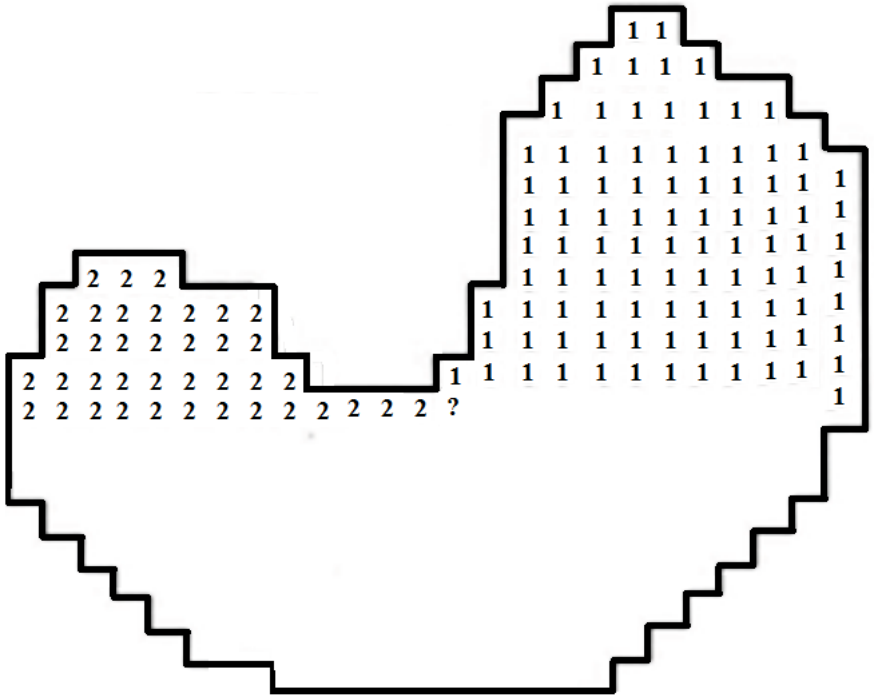


## 2.3 Segmenting Binary Images

**Second algorithm: Sequential labeling**

However, what do we do if D belongs to the background, B and C are labeled, but they have different labels? For the object shown here, this exact case arises for the pixel shown as a black box.

Here, pixel D belongs to the background (it has the value 0) pixel B has the label 1 and pixel C has the label 2.



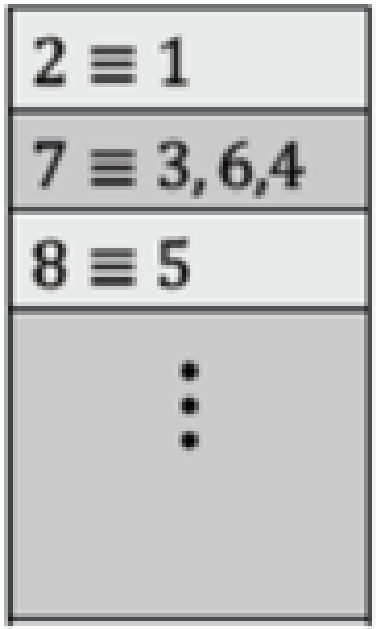
# Chapter 2. Binary images

## 2.3 Segmenting Binary Images

In this case, we assign pixel A the label of either B or C and simply make note of the fact that the label of B is “equivalent” to the label of C.

This information is stored in an equivalence table. This table can be made compact by representing each set of equivalent labels by a single label.

After the image has been fully scanned and labeled, we simply do a second raster scan of the image where the equivalences are resolved.

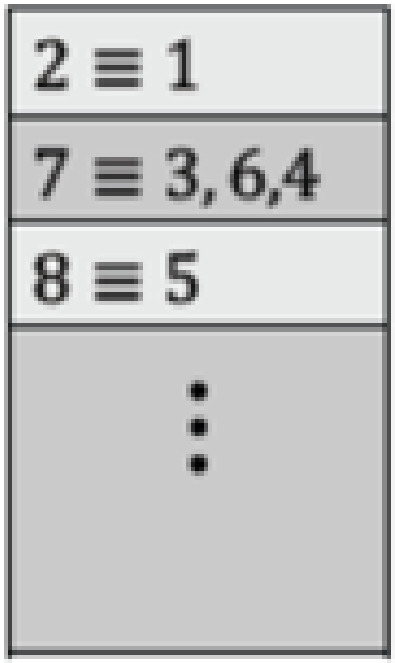


# Chapter 2. Binary images

## 2.3 Segmenting Binary Images

In summary, with just two raster scans of the image, it is fully segmented into distinct regions, or objects.

Now the geometric properties (moments) we defined previously can be computed for each of the objects in the image in order to recognize it and find its position and orientation.



# Chapter 2. Binary images

## 2.4 Iterative Modification

# Chapter 2. Binary images

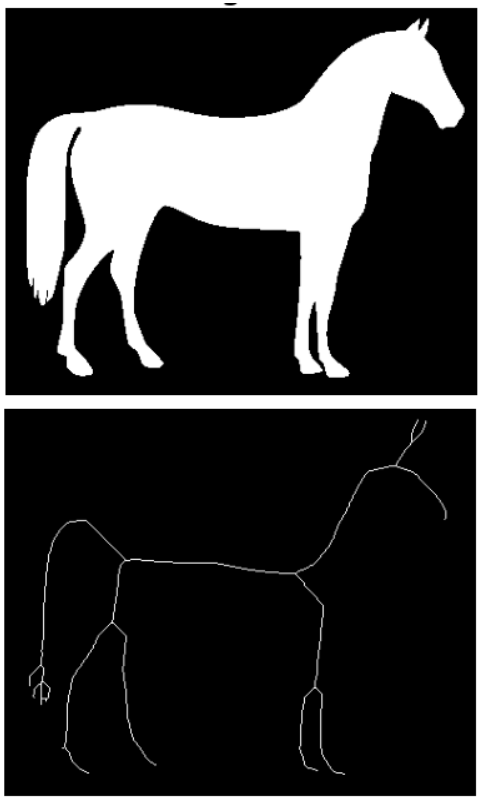
## 2.4 Iterative Modification

**The skeleton**.
A binary image can be iteratively modified to extract useful information from it

Given a binary image of a full human body, one could extract the skeleton of the body by thinning the body in the image without compromising the overall structure.

The skeleton could be used to determine information about the body, such as its pose.

The skeleton is usually computed via iterative thinning, distance transforms, or morphological operations.



# Chapter 2. Binary images

## 2.4 Iterative Modification

**The skeleton**

**The Euler number:**

The Euler number (or Euler characteristic) is a topological invariant that describes the structure or connectivity of a binary image. It is defined as: the number of bodies minus the number of holes.


The Euler number of the full image is the sum of the Euler numbers of all its non-overlapping regions.

Letter B: $E_b = -1$

Letter i: $E_i = 2$

Letter n: $E_n = 1$

Image : $E = 4$

$$E_{image} = \sum E_{non-overlapping\ regions}$$

If an operation is applied to a region of the image and it does not change the Euler number of the region, then the Euler number of the complete image will remain the same.

# Chapter 2. Binary images

## 2.4 Iterative Modification

**The skeleton**

**Link Between Euler Number and Skeleton Computation**

**Topological Consistency**: When computing the skeleton of a shape, the Euler number remains an important measure to ensure that the topological features of the shape are preserved.
Skeletonization typically aims to maintain the number of connected components and holes of the original object. Therefore, the Euler number before and after skeletonization should ideally remain the same.


Letter B: $E_b = -1$

Letter i: $E_i = 2$

Letter n: $E_n = 1$

Image : $E = 4$

$$E_{image} = \sum E_{non-overlapping\ regions}$$

# Chapter 2. Binary images

## 2.4 Iterative Modification

**The skeleton**

**Link Between Euler Number and Skeleton Computation**

**Thinning and Euler Number**: In skeleton computation via iterative thinning, local operations are used to progressively reduce the shape while preserving its topology. If the thinning process alters the Euler number, it may indicate a loss of topological features such as the disappearance of components or holes, which is undesirable. Thus, the Euler number can be monitored to ensure that the thinning algorithm preserves the shape's topology.


Letter B: $E_b = -1$

Letter i: $E_i = 2$

Letter n: $E_n = 1$

Image : $E = 4$

$$E_{image} = \sum E_{non-overlapping\ regions}$$

# Chapter 2. Binary images

## 2.4 Iterative Modification

**The skeleton**

**Link Between Euler Number and Skeleton Computation**

**Pruning and Noise Handling**: When computing skeletons, noisy objects or artifacts in the image can affect the Euler number (e.g., creating small unwanted components or holes). Pruning techniques may be applied to the skeleton to clean up these artifacts, while checking the Euler number to avoid removing important topological features.


Letter B: $E_b = -1$

Letter i: $E_i = 2$

Letter n: $E_n = 1$

Image : $E = 4$

$$E_{image} = \sum E_{non-overlapping\ regions}$$

# Chapter 2. Binary images

## 2.4 Iterative Modification

**The Euler number:**
Euler differential as the change in the Euler number of an image due to an operation applied to it.

We will assume, for convenience, that the image is a hexagonal grid.

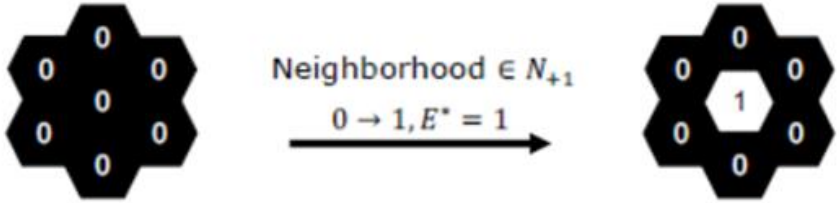
For the image of the figure, the Euler differential is 1 minus 0, which is 1.

# Chapter 2. Binary images

## 2.4 Iterative Modification

**The Euler number:**

On a hexagonal grid, each pixel has 6 neighbors and hence 64 possible neighborhood patterns (patterns of 0s and 1s).
These patterns can be classified based on the Euler differential they generate when the center pixel goes from a 0 to a 1.




In this pattern, if the center pixel is changed from 0 to 1, the Euler differential is 1. We say that it belongs to the neighborhood class $N_{+1}$.

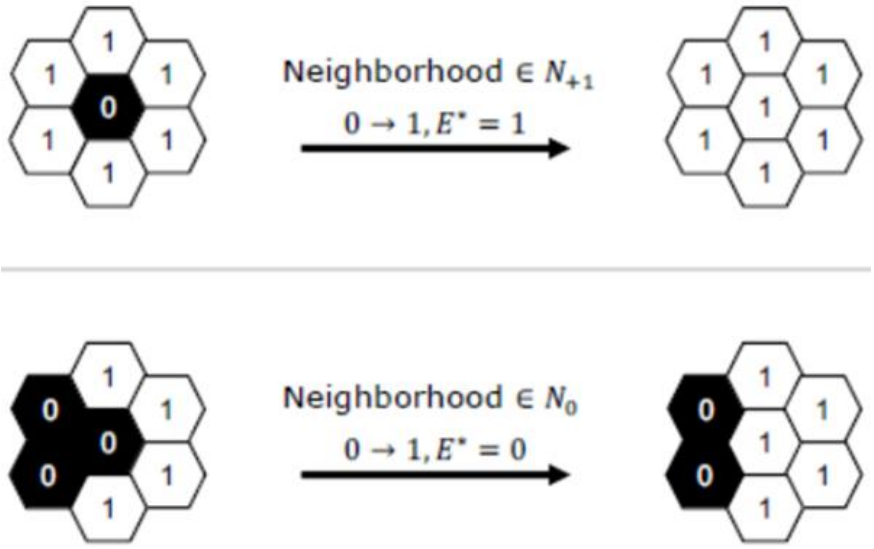
# Chapter 2. Binary images

## 2.4 Iterative Modification

**The Euler number:**

Here is another example that belongs to the neighborhood class $N_{+1}$.

Below it we have a pattern that belongs to the neighborhood class $N_0$, which is important class because it represents conservative operators — these are neighbourhood patterns for which the center pixel value can be modified without introducing any new bodies or holes in the pattern and hence in the larger image it belongs to.

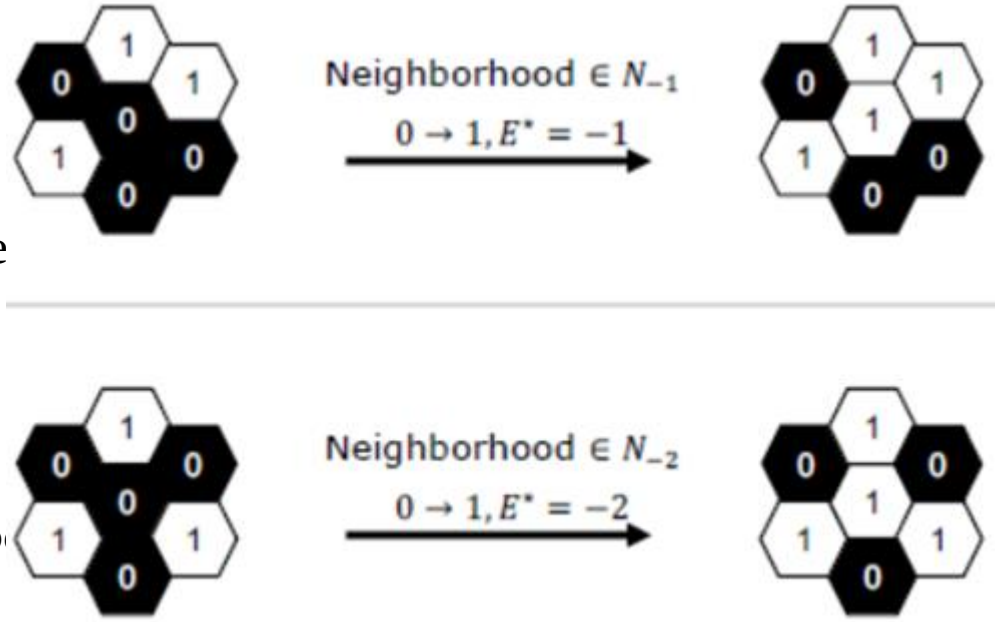


# Chapter 2. Binary images

## 2.4 Iterative Modification

**The Euler number:**

Here is a case where the Euler differential is -1. When the center pixel is changed from a 0 to a 1, two bodies are connected, causing the number of bodies to decrease by one On the bottom, a pattern is shown that belongs to the neighbourhood class $N_{-2}$.




Upon examining all 64 neighborhoo patterns, we find that there are only four possible neighborhood types: $N_{+1}$, $N_0$, $N_{-1}$, and $N_{-2}$.
We would use an operation that is only applicable to the $N_0$ class of neighborhoods

# Chapter 2. Binary images

## 2.4 Iterative Modification

Images of figure below show the skeletons obtained by iteratively applying the thinning algorithm to binary images of a butterfly and a human body.

# Chapter 2. Binary images

**References**

[1] Robot Vision. B.K.P. Horn. MIT Press. McGraw-Hill, LLC.